```
  hour = bcdToDec(Wire.read());
  weekday = bcdToDec(Wire.read());
  monthday = bcdToDec(Wire.read());
  month = bcdToDec(Wire.read());
  year = bcdToDec(Wire.read());
}
```

Source Code Summary

This one sketch might seem overwhelming so let's focus on our first sketch in Listing 10-3 before we talk about the following version in Listing 10-4. I promise that it's not too complicated. The code has been broken down into functions, so it's not too hard to understand what each chunk of code does. To start things off, the manufacturer of every I2C device will assign it a unique address for that device. Rather than referring to the hexadecimal value `0x68`, a rather confusing number for the address of our RTC, we assign the value a name that relates to our device, or in the following case `DS1307`:

```
const int DS1307 = 0x68;
```

We then set up a pair of character arrays to contain text strings for both the names of the days of the week and the months of the year. We also have a block of byte data type variables where we will assign data for seconds, minutes, hours, and so on.

Our `setup()` function uses the `Wire.begin()` and `Serial.begin()` functions to start both of these communication protocols, one for talking to the RTC and the other for talking to our computer. The `loop()` function shows one way that an RTC might be useful in place of long delays and in fact, there are no delays in the `loop()` function at all. We don't even need to display the time, instead we could have an event occur every 5 minutes, every hour on the hour, or even once daily at 7:00 a.m., like a cuckoo clock. We do this by reading the data from the RTC and comparing one of the values against a condition. For example, we decided to only update our time every minute, as follows:

```
if (minute != lastMinute) {
```

Here we are checking to see if the minute has changed or a minute has passed before reprinting the current time to the Serial Monitor. This is only one small way of using the RTC as a timer and there could be many, many more. It has the advantage of being very precise and because of the battery backup, is reliable even when the power is off. So let's look at the two functions called in the `loop()` function of our sketch.

The first function that we call is `readTime()`, which is used to read the date and time data from our I2C device. This function quite simply starts the communication with the assigned device and then reads the 7 bytes of data that is sent from the device. This is performed in a particular sequence according to the device. The `readTime()` function also calls another function called `bcdToDec()`, which nicely converts the BCD-formatted data to decimal-formatted data that we can better put to use.

The second function called is `printTime()`, which takes six of the seven time measurements and formats them for printing to the screen. This function jumps through some hoops that are not entirely necessary, but make the time a little easier to read. This includes fudging the array position a little, the RTC starts at 1 while our arrays start at 0, so we subtract one from the array position to get the names for the days and months, converting the 24-hour clock to 12-hour, and then buffering the minutes value with another one of those C functions, `sprintf()`, to create leading zeros so that instead of a time reading of 12:4 we would get the more proper display of 12:04.

So now let's look at the next sketch in Listing 10-4 to see how we can go about setting the time in our real-time clock.